

Lecture 1

Lecture 1:

Why Deep Learning & Why Now?

Traditional Machine Learning typically defines sets of features. Features are basically rules of how to do a task step by step. Usually we as humans aren't too good at defining robust features and therefore machine learning finds good features to use for a task.

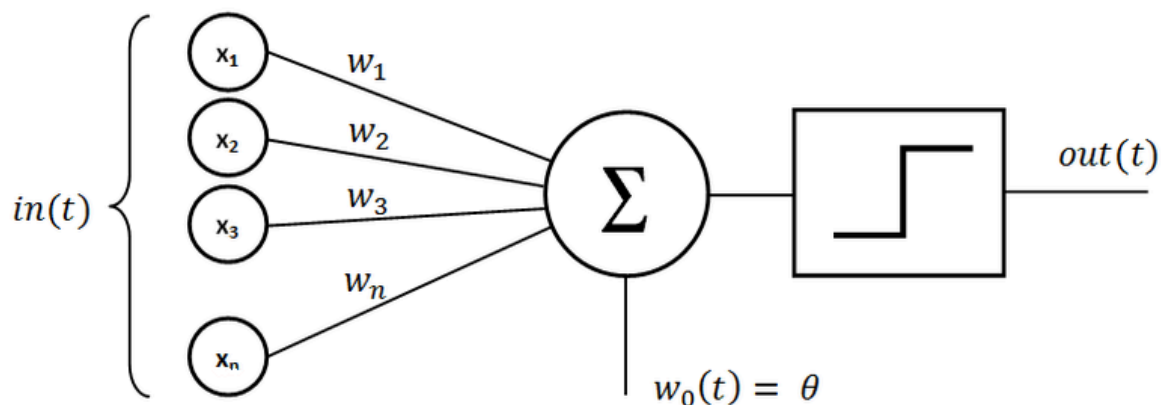
Deep learning observes and creates its own features to classify or predict data rather than being explicitly told about the features. Deep learning learns to hierarchically detect objects in an order of low-level features to high level features where it takes small low level features and composites them together to create higher level features.

Additionally, now there is dominance of neural networks because of big data sets, easier collection and storage of data sets, graphics processing units and good hardware in general and lastly, software has improved with better libraries and models to create neural networks.

Fundamentals of Neural Networks

The perceptron or neuron is the fundamental and structural building block of deep learning.

Perceptron takes multiple inputs, with each having its own weight, the inputs are multiplied by their weights and summed together with an additional value called the 'bias', a shifting scalar to finally be passed through a non-linear one-dimensional function also called the activation function to create the output. Here's a diagram of the perceptron:



The perceptron equation:

$$\hat{y} = g \left(w_0 + \sum_{i=1}^m x_i w_i \right)$$

Here is a breakdown of the equation:

- The y-hat symbol on the left means the predicted value for the output
- On the right hand side, the g refers to the nonlinear/activation function
- Inside the activation function, the w value refers to the bias of the function
- The sigma function adds all of the products of each input and their respective weights.

The activation function

Activation functions are used to introduce non-linearities to models because the real world data is often nonlinear and therefore nonlinearities are required to create useful neural networks. The benefit of having such non-linearities in models is that it allows it to approximate arbitrarily complex functions with enough depth in the model– making them extremely powerful.

Some common activation functions include the sigmoid function, the hyperbolic function, the rectified linear unit (ReLU)

NOTE: Sigmoid function is good for converting outputs into probabilities

Using multiple perceptrons

Complex neural networks have multi-output perceptron layer(s). Such layers are called 'hidden layers,' simply because those aren't visible, only the input to a model and it's output is visible.

NOTE: Dense layers are layers where every perceptron is connected to every input.

A 'deep neural network' is one with multiple hidden layers where multiple nonlinearities and functions stack upon each other to produce the output. In such cases the input for any layer come from the previous layer and the first layer is always the input and the last layer is always the output. It is a hierarchical combination of going deeper into linear and non-linear operations.

More layers in x-axis = More depth and complexity because more hierarchical functions are introduced. For example, predicting the price of a stock based on recent news and stock graphs would require more depth as compared to multiple outputs.

More layers in y-axis = More outputs produced. For example, producing an image requires multiple pixels.

Training Neural Networks

Vocab:

- Loss= The loss of a neural network is a quantified form of how far from correct an output is, or, how 'good' or 'bad' the output is.
 - Smaller loss = Close prediction
 - Higher loss= Far/Poor prediction
- Empirical Loss= Measure of the total loss of a model over the entire training dataset
- Mean Squared Error Loss (MSE Loss)= Can be used with regression models that output continuous real numbers.
 - Measured based on the absolute difference between prediction

NOTE: Neural networks are punished based on error to allow them to train and learn.

Goal of training datasets: Minimize the empirical loss (Loss Optimization)

A multidimensional plot of the weights for each input and the empirical loss of the dataset can show which pair of points produces the lowest empirical loss.

ONE method for loss optimization is 'Gradient Descent'.

Gradient descent algorithm:

1. Initialize weights randomly (Random place on the graph)
2. Loop until convergence (Repeat 2a and 2b until model stops improving):

a.

Compute gradient, $\frac{\partial J(W)}{\partial W}$

- i. Gradient is the direction of steepest ascent (where the model is being inaccurate)
- ii. $J(W)$ is the loss function
- iii. Reveals how much of a change in error will be caused by a change in weights

- b. Update weights

$$W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$$

- i. Gradient points in the direction of higher loss, it is subtracted to go in the direction of lower loss
- ii. "Eta" symbol represents step size (If too high, model may overshoot, if too low, will be way slower)

3. Return Weights (Returns "trained" weights)

Stochastic Gradient descent is a similar technique, but rather than computing the weights over the whole dataset, a noisy gradient is going to be computed.

Algorithm:

1. Initialize weights randomly
2. Loop until convergence:
 - a. Pick single data point or 'mini-batch' of (A small portion of the dataset) i

Compute gradient, $\frac{\partial J(W)}{\partial W}$

- b.

$$W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$$

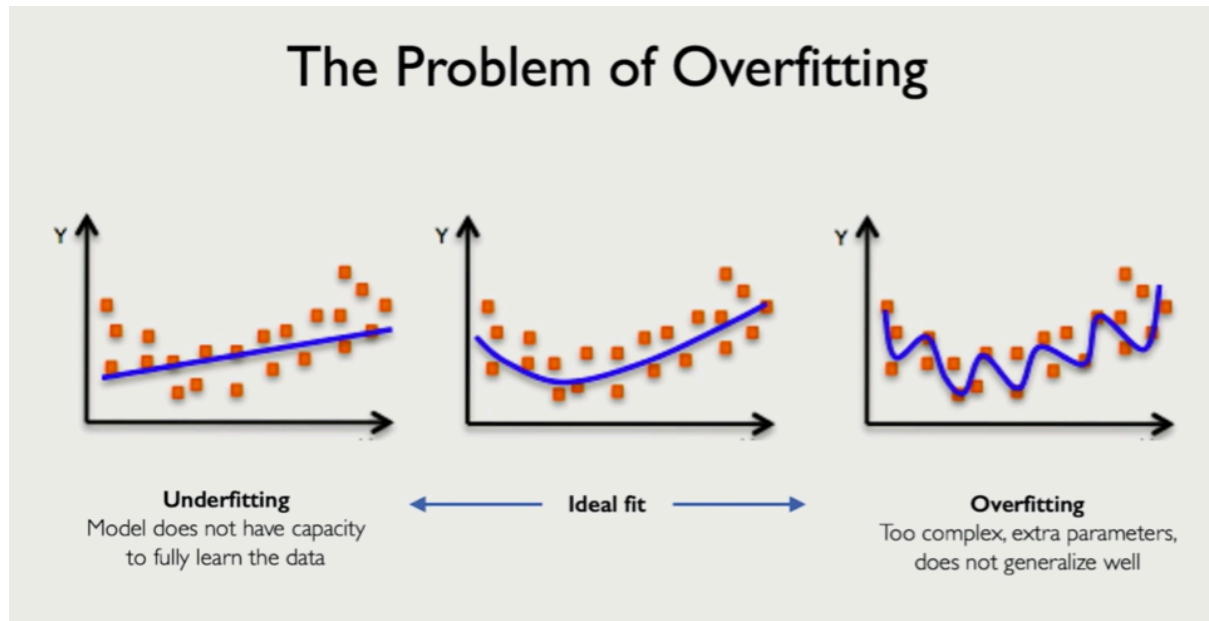
- c. Update weights

3. Return weights

Using mini batches rather than singular data points allows for the neural network to be robust enough to train, but prevents high computation needs. Using mini batches prevents noisy gradients that may occur when picking random data points, leading to mini batches allowing for smoother convergence (improvement of neural network) and allows for larger learning rates.

Parallelizing computation, i.e, using multiple computers parallelly to train neural networks with separate mini-batches to allow for models to rapidly improve.

Neural Networks in Practice: Overfitting



While training models, it is important that the neural network generalizes well enough to not only work with training dataset, but also real-life dataset. Overfitting can lead to too complex neural networks that may form biases and parameters that may not fit well into real-life data, rather it may create parameters that may only work for training datasets.

Regularization is a technique that constrains the optimization problem to discourage complex models; it improves generalisation of the model so that it works well on unseen data.

Regularization 1: Dropout

During training, they randomly set some activation to 0, this forces the network not to rely so much on the output of a neuron and prevents the model from memorising, and rather forces it to generalize.

It is a very powerful method and is the most popular form of regularization.

Regularization 2: Early Stopping

Stop training before we have a chance to overfit. As the model trains, after iteration, testing data loss nad training data loss is measured, whenever training loss starts to increase, training should be stopped before it does, to prevent overfitting.

Lecture 2

Lecture 2:

Sequential Modeling

In many applications, the neural networks need to model their outputs by processing data one after another. For example, when trying to complete a sentence, the words need to be processed as a sequence rather than as multiple data points, sequential modeling will be used to process the data as a sequence.

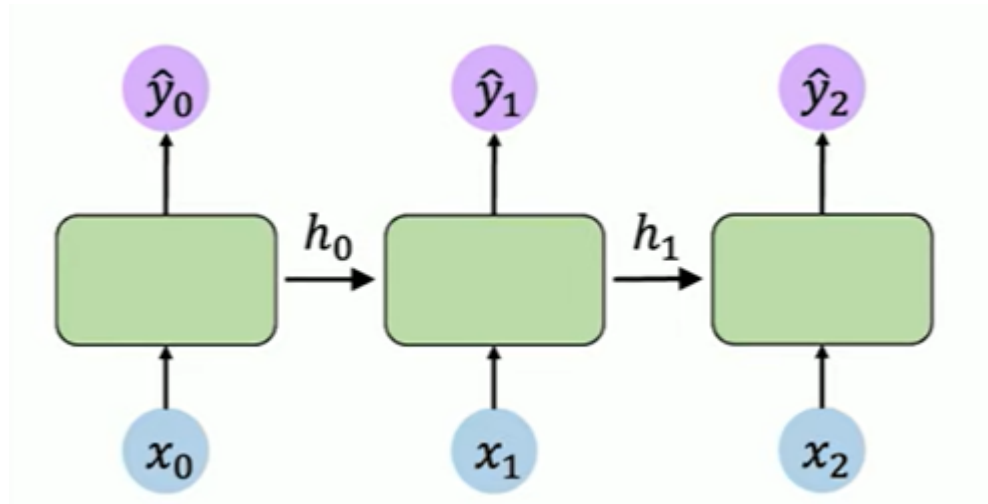
Some examples of datasets that can be processed as a sequence: motion of objects, market trajectories, stocks, videos, audio, text and more are all very common types of inputs for modern AI applications.

This is not only used to process inputs, but also to create outputs like sentences, audio, etc.

Sequential modeling can be in multiple input-output formats including one to many, many to one and many to many.

Neurons with Recurrence

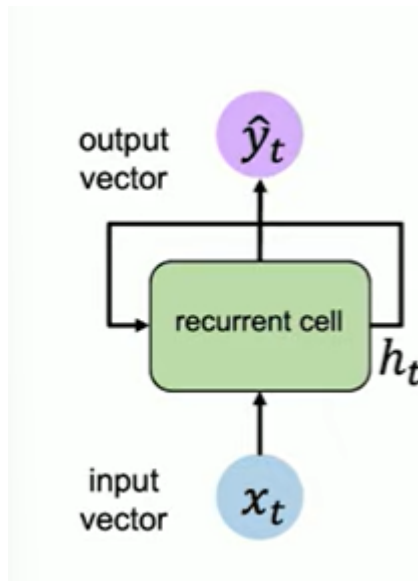
While feed-forward networks, which are networks that feed every input to every output, they lack recurrence needed for such sequential modeling. To make them recurring, multiple of such networks need to be placed one after another for every data point in the sequential input or output, the output of the previous feed-forward network can be connected to the next feed-forward network as an additional input so that it factors into the output of the network.



This is a diagram that represents a many to many sequential neural network, where by hooking up the output of the previous neural network to the next one, this allows for the model to have recurrence. Here, the carried over notion of the internal state of the network is denoted by 'h'. Using these variables presented in the diagram, we can create an equation to model this recurrence:

$$\hat{y}_t = f(x_t, h_{t-1})$$

output input past memory



Such a recurrent cell can be represented by this diagram:
This diagram represents the same form of recurrence, where the input goes through a recurrent cell with h of t (looping through all the inputs) to produce the output.

Recurrent Neural Networks (RNNs)

Such neural networks are classified as RNNs, where this recurrence occurs using an updating state ' h '. Here is the same equation from above with labelled variables.

$$\boxed{h_t} = \boxed{f_W}(\boxed{x_t}, \boxed{h_{t-1}})$$

cell state
function
input
old state

with weights W

Expanding on this, there is another equation to update the ' h ' (hidden) state.

$$h_t = \tanh(W_{hh}^T h_{t-1} + W_{xh}^T x_t)$$

Here, the weight matrices are multiplied to both, the previous hidden value and the current input for the perceptron out of the sequence of inputs